



Offchain Labs OSP Soundness Guard

Security Assessment (Summary Report)

September 15, 2026

Prepared for:

Harry Kalodner, Steven Goldfeder, and Ed Felten

Offchain Labs

Prepared by: **Simone Monica and Jaime Iglesias**

Table of Contents

Table of Contents	1
Project Summary	2
Project Targets	3
Executive Summary	4
Summary of Findings	6
Detailed Findings	7
1. Guard trigger will fail if latestConfirmed changes	7
2. Scripts contain references to previous contract versions	9
A. Vulnerability Categories	10
B. Code Quality Recommendations	12
About Trail of Bits	13
Notices and Remarks	14

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Simone Monica , Consultant simone.monica@trailofbits.com	Jaime Iglesias , Consultant jaime.iglesias@trailofbits.com
---	--

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 10, 2026	Pre-project kickoff call
August 20, 2026	Delivery of report draft
September 15, 2026	Delivery of final summary report

Project Targets

nitro-contracts-private

Repository	https://github.com/OffchainLabs/nitro-contracts-private
Version	12204ae505b3d9440e134f721d3c15dc97a5b049
Type	Solidity
Platform	EVM

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review the security of the BOLD OSP Soundness Guard contract.

The OSP Soundness Guard contract is a circuit breaker for BOLD, allowing the executor to pause the challenge protocol. The guard can be triggered only under certain circumstances: specifically, when two rival length-one SmallStep edges can both be one-step-proven, thereby identifying a soundness issue.

A team of two consultants conducted the review from August 12 to August 17, 2026, for a total of eight engineer-days of effort. With full access to source code and documentation, we performed static and dynamic testing of the target, using automated and manual processes.

Observations and Impact

The main focus of the review was ensuring that the implementation respects the design, is implemented correctly, and does not introduce unwanted edge cases. Furthermore, we verified a number of key invariants, such as the following:

1. The only non-view external call the guard makes to a contract it does not own is `pauser.pause(address(rollup))`, and `rollup` is immutable.
2. The `UpgradeExecutor` is never made to do anything other than call the `pause()` selector with a zero value. It does not invoke `delegatecall`, call any other selector, or perform value forwarding.
3. Every read the guard performs from the caller-supplied `cm` is a view function, so `solc` emits `STATICCALL` and a hostile manager cannot write state or reenter with effect.
4. A pause requires two distinct rival edges that both pass the inherited confirmation, in separate stores.
5. A pause requires the edges' `prev` assertion to descend from the rollup's current `latestConfirmed` assertion.
6. `provenDescendsFrom` entries only ever move from `false` to `true`, and a walk follows only the links the caller lists.
7. All shadow writes land in a clone created during that same transaction.
8. Neither the guard nor `PauseExecutor` is payable, and neither has a `receive` or `fallback`, so no ETH can enter or leave.

Note that issues related to privileged actors, such as governance or role holders, were specifically out of scope for the review (e.g., issues such as a PAUSER_ROLE holder behaving maliciously or governance misconfiguring a contract).

Our review resulted in two informational-severity findings.

Recommendations

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Guard trigger will fail if latestConfirmed changes	Data Validation	Informational
2	Scripts contain references to previous contract versions	Configuration	Informational

Detailed Findings

1. Guard trigger will fail if latestConfirmed changes

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OSPGUARD-1

Target: `src/challengeV2/guard/OspSoundnessGuard.sol`

Description

Before pausing the rollup, the guard requires the caller to prove that the challenged fork is still live. `_requireDescendantOfLatestConfirmed` does this by walking the assertion chain upward from the fork point, and it requires the walk to end exactly on `rollup.latestConfirmed()`.

```
function _requireDescendantOfLatestConfirmed(
    bytes32 prevAssertionHash,
    bytes32[] calldata checkpoints,
    AssertionAncestryStep[] calldata ancestry
) internal view {
    bytes32 cursor = prevAssertionHash;
    for (uint256 i = 0; i < checkpoints.length; i++) {
        if (!provenDescendsFrom[cursor][checkpoints[i]]) {
            revert AncestryLinkNotProven(cursor, checkpoints[i]);
        }
        cursor = checkpoints[i];
    }
    bytes32 latestConfirmed = rollup.latestConfirmed();
    if (_walkUp(cursor, ancestry) != latestConfirmed) {
        revert NotDescendantOfLatestConfirmed(prevAssertionHash, latestConfirmed);
    }
}
```

Figure 1.1: The `_requireDescendantOfLatestConfirmed` function
(`OspSoundnessGuard.sol`#L169–L185)

The walk is fixed in the caller's calldata, but `latestConfirmed` is read at execution time. While a challenge is open, `latestConfirmed` trails the fork point by every assertion created in the last `confirmPeriodBlocks`, and it advances each time one of those assertions is confirmed.

Since `minimumAssertionPeriod` is 75 blocks, this occurs repeatedly over the life of a challenge, as a result of ordinary validator activity.

If `latestConfirmed` advances before the transaction is executed, the walk passes through the new value and terminates on the previous one. The comparison then fails, and the transaction reverts.

Recommendations

Short term, document this scenario so that it does not cause confusion when triggered. It is unlikely to be triggered and requires only a new transaction to be sent to overcome it.

2. Scripts contain references to previous contract versions

Severity: Informational

Difficulty: High

Type: Configuration

Finding ID: TOB-OSPGUARD-2

Target: scripts/deploy0spSoundnessGuard.ts

Description

The OSP guard deployment script contains two identifiers left over from renaming Pauser to PauseExecutor. These leftover identifiers cause two failures. The first occurs before any contract is deployed, so the second failure is unreachable until the first is fixed.

First, the script requests a nonexistent Pauser artifact instead of PauseExecutor (figure 2.1). Therefore, Hardhat fails before deploying any contracts.

```
const pauser = await deployContract(
  'Pauser',
  deployer,
  [target.upgradeExecutor, target.upgradeExecutor, []],
  true,
  true
)
```

*Figure 2.1: Deployment using a nonexistent contract artifact
(deploy0spSoundnessGuard.ts#L87-L97)*

Once the artifact name is corrected, the script reaches its call to `guard.pauser()`. The guard exposes the pause executor through the autogenerated `pe()` getter, so this call also fails. It occurs after deployment but before the script prints the governance calldata needed to install the guard.

```
if ((await guard.pauser()).toLowerCase() !== pauser.address.toLowerCase()) {
  throw new Error('guard points at a different pauser')
}
```

*Figure 2.2: Sanity check using a nonexistent getter
(deploy0spSoundnessGuard.ts#L115-L122)*

Recommendations

Short term, replace Pauser with PauseExecutor and `guard.pauser()` with `guard.pe()`.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

The following recommendations are not associated with any specific vulnerabilities. However, implementing them will enhance code readability and may prevent the introduction of vulnerabilities in the future.

- **Check that the recorded challenge manager belongs to the guarded rollup.**
After the config-hash check has pinned the caller-supplied manager, require `address(cm.assertionChain()) == address(rollup)`. This could be done as part of the OSP installation.
- **Restrict or clearly document the PAUSER_ROLE.**
`PauseExecutor.pause` accepts a caller-supplied target, so a role holder can request `pause()` on any contract for which the upgrade executor is authorized, not only the guard itself. Role holders are assumed to be trusted, but it is worth highlighting this behavior.
- **Update tests to include a multilevel challenge to verify that all the logic is correctly reproduced.**

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Offchain Labs under the terms of the project statement of work and has been made public at Offchain Labs' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.